

# Arquitectura del Software

Jesús Sánchez Cuadrado

Última actualización: 05/05/2026

Este tema se introduce el concepto de arquitectura de software y su importancia en el desarrollo de sistemas. Se presentarán algunas arquitecturas.

Luego se desarrollarán algunos tipos de arquitecturas comunes en el desarrollo de software:

- REST
- Arquitectura hexagonal / Arquitectura limpia
- Colas de mensajes

## 1. Introducción

El desarrollo de sistemas software en la actualidad implica enfrentarse a problemas muy diversos, que dependen en gran medida del tipo de aplicación que se esté construyendo.

En un videojuego online, por ejemplo, es necesario gestionar correctamente la sincronización entre jugadores, la latencia de red y el estado compartido en tiempo real, de manera que todos los usuarios perciban una experiencia coherente. En sistemas bancarios, en cambio, el foco está en garantizar la consistencia y la fiabilidad de las operaciones, evitando pérdidas o duplicidades en las transacciones. En plataformas de comercio electrónico como Amazon, especialmente en momentos de alta demanda como el Black Friday, el reto principal es la escalabilidad, permitiendo atender a millones de usuarios simultáneamente sin degradar el rendimiento del sistema. Por otro lado, en aplicaciones de mensajería, es fundamental asegurar la entrega de mensajes, su orden y su persistencia, incluso en escenarios donde los usuarios no están conectados.

Todos estos problemas tienen en común que no pueden resolverse únicamente a nivel de implementación, sino que requieren tomar decisiones sobre cómo estructurar el sistema, cómo dividirlo en partes y cómo deben comunicarse entre sí.

Estas decisiones forman parte de la **arquitectura del software**, que proporciona un marco para diseñar sistemas capaces de responder a este tipo de requisitos de manera eficiente y mantenible. En este tema se introducirá el concepto de arquitectura del software y se analizarán algunos de los principales estilos arquitectónicos utilizados en la práctica.

## 2. Arquitectura del software

La **arquitectura del software** es la estructura fundamental de un sistema de software, compuesta por sus componentes, las relaciones entre ellos

## Índice

|                                                      |    |
|------------------------------------------------------|----|
| 1. Introducción .....                                | 1  |
| 2. Arquitectura del software .....                   | 1  |
| 2.1. Definiciones de arquitectura del software ..... | 2  |
| 3. Partes de la arquitectura .....                   | 4  |
| 4. Estilos arquitectónicos .....                     | 4  |
| 4.1. Definición .....                                | 5  |
| 4.2. Arquitectura en capas .....                     | 5  |
| 4.3. Microkernel .....                               | 6  |
| 4.4. Pipes and filters .....                         | 7  |
| 4.5. Arquitectura basada en eventos ..               | 8  |
| 4.6. Arquitectura cliente-servidor .....             | 8  |
| 4.7. Microservicios .....                            | 9  |
| 5. Arquitectura hexagonal .....                      | 9  |
| 5.1. Elementos .....                                 | 10 |
| 5.2. Ejemplo de arquitectura hexagonal y DDD .....   | 11 |
| 5.3. Ventajas y desventajas .....                    | 14 |
| 6. Arquitectura REST .....                           | 15 |
| 6.1. El protocolo HTTP .....                         | 16 |
| 6.2. La arquitectura REST .....                      | 20 |
| Bibliografía .....                                   | 23 |

y los principios y decisiones que guían su diseño y evolución. El objetivo es poder describir y razonar sobre un sistema a alto nivel.

La arquitectura del software no se centra en los detalles de implementación, sino en las decisiones fundamentales que afectan al sistema en su conjunto, y describe cómo está organizado el software y cómo interactúan sus partes para cumplir los requisitos no funcionales (rendimiento, seguridad, escalabilidad, mantenibilidad, etc.).

En la arquitectura el foco ya no son unidades pequeñas del código (ej., clases) sino que el foco está puesto en unidades más grandes (ej., módulos, servicios, capas, componentes). Por tanto, la arquitectura tiene una granularidad mayor que el diseño de software.

Algunos de los elementos implicados en la arquitectura de software son:

- Paquetes
- Módulos
- Componentes
- Capas
- Servicios
- Sistemas y subsistemas

## 2.1. Definiciones de arquitectura del software

Existen multitud de definiciones de arquitectura del software, más o menos formales. A continuación se presentan algunas de las más conocidas.

### 2.1.1. Basada en las relaciones entre los elementos

La arquitectura del software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema, las cuales comprenden los elementos de software, las relaciones entre ellos y las propiedades de ambos.”

– Software Architecture in Practice, Bass, Clements y Kazman (2013)

### 2.1.2. Basada en las decisiones principales

“La arquitectura del software trata de las decisiones importantes, aquellas que son difíciles de cambiar.”

– Philippe Kruchten

“Architecture is about the important stuff. Whatever that is.”

– Ralph Johnson

Esta definición pone el foco en la arquitectura refleja las decisiones críticas del proyecto, aquellas que deben ser correctas desde el principio del proyecto.



*“Architecture is about the important stuff. Whatever that is.”*

Ralph Johnson  
(Coautor de “Design Patterns”, GoF)

Esencialmente, la arquitectura del software se refiere a las decisiones fundamentales sobre la estructura y organización de un sistema de software y el impacto de estas decisiones.

### 2.1.3. Rol de la arquitectura

La **arquitectura del software** es el conjunto de estructuras necesarias para razonar sobre un sistema software (a alto nivel). Cada estructura está formada por elementos software, relaciones entre ellos y ciertas propiedades de los elementos y relaciones. Estas estructuras principales reflejan las decisiones críticas del proyecto, aquellas que deben ser correctas desde el principio del proyecto.

De alguna forma, se puede decir que la arquitectura son “Los planos del software”, que incluyen:

- Estructura
- Comportamiento
- Interacciones
- Propiedades no funcionales

En particular, es importante destacar que la arquitectura del software tiene como objetivo abordar los requisitos no funcionales. La razón es que los requisitos funcionales se pueden abordar agregando, modificando o eliminando código. Sin embargo, los requisitos no funcionales suelen ser “cross-cutting concerns”, esto es, afectan a múltiples partes del sistema. Por tanto, requieren una planificación y diseño cuidadosos a nivel arquitectónico para garantizar que se cumplan de manera efectiva en todo el sistema.

### 2.1.4. Diseño vs. Arquitectura

En el desarrollo de software, la arquitectura y el diseño son conceptos estrechamente relacionados, pero no equivalentes. Ambos forman parte del proceso de construcción de un sistema, aunque operan a distintos niveles de abstracción.

Por un lado, la arquitectura define la estructura general del sistema; es decir, cómo se organizan sus componentes principales, cómo se comunican entre sí y qué decisiones globales condicionan su desarrollo y evolución. Por otro lado, el diseño se centra en los detalles internos de cada uno de esos componentes, como la estructura de las clases, los algoritmos utilizados o la organización del código.

Una diferencia importante entre arquitectura y diseño es el coste de cambio. Las decisiones arquitectónicas son, en general, más difíciles y costosas de modificar una vez que el sistema ha sido implementado, ya que afectan a la organización global del sistema. Por el contrario, las decisiones de diseño suelen ser más locales y, por tanto, más fáciles de ajustar sin impactar en el resto del sistema.

Se puede decir que *cualquier decisión arquitectura es un tipo de decisión de diseño*, pero no necesariamente al revés. La Fig. [Figura 1](#) muestra esta situación. La siguiente tabla, tomada de [\[1\]](#), describe el ámbito de la arquitectura, el diseño y la implementación. En particular, la arquitectura es un tipo de especificación abstracto porque no entra en detalles relativos a cómo se implementa el código, al igual que el diseño, pero el ámbito de la

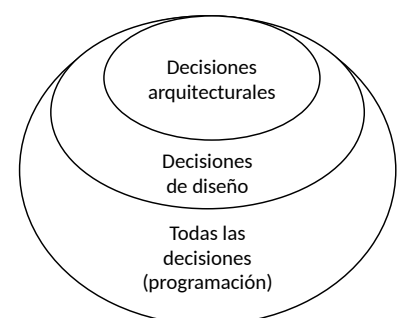


Figura 1: Comparación de diseño y arquitectura

arquitectura es el sistema completo (esto es, tiene en cuenta el impacto de cada componente en las propiedades globales del sistema), mientras que el diseño se centra en componentes individuales.

|                | Tipo de especificación | Ámbito             |
|----------------|------------------------|--------------------|
| Arquitectura   | Abstracto              | Sistema completo   |
| Diseño         | Abstracto              | Partes del sistema |
| Implementación | Concreto               | Partes del sistema |

### 3. Partes de la arquitectura

En cualquier sistema software, independientemente de su tamaño o complejidad, es posible identificar dos elementos fundamentales: los **componentes** y los **conectores**.

Un **componente** es una unidad funcional del sistema que encapsula parte de su comportamiento, como un módulo, un servicio o una capa.

Los componentes representan las distintas partes en las que se divide el sistema, cada una con una responsabilidad concreta. Por ejemplo, en una aplicación web podemos encontrar componentes encargados de la gestión de usuarios, el procesamiento de pedidos o el acceso a datos.

Un **conector** es el mecanismo que permite la comunicación entre componentes, gestionando el intercambio de información entre ellos.

Los conectores determinan cómo interactúan los componentes entre sí. Esta interacción puede materializarse de distintas formas, como llamadas a funciones, invocaciones a APIs, intercambio de mensajes o eventos.

Desde esta perspectiva, un sistema software puede entenderse como un conjunto de componentes que procesan información y conectores que permiten que dicha información fluya entre ellos.

El diseño de la arquitectura implica, por tanto, decidir qué componentes son necesarios y cómo deben comunicarse. Estas decisiones tienen un impacto directo en propiedades del sistema como la modularidad, la escalabilidad o la mantenibilidad.

### 4. Estilos arquitectónicos

A continuación se presentan algunos estilos arquitectónicos comunes en el desarrollo de software. Cada estilo tiene sus propias características, ventajas y limitaciones, y la elección de uno u otro dependerá del contexto del proyecto y de los objetivos que se quieran alcanzar.

Para completar esta explicación se recomienda consultar los siguientes recursos:

- [2], que presenta un catálogo de estilos arquitectónicos con ejemplos y casos de uso.
- [3] (capítulo 7), describe en detalle la arquitectura en capas y la arquitectura cliente-servidor.
- [4] (capítulo 7; <https://softengbook.org/chapter7>), incluye referencias a varios estilos incluyendo arquitectura basada eventos y microservicios.

#### 4.1. Definición

A lo largo del tiempo, se han identificado distintos **estilos arquitectónicos**, que representan formas generales de organizar sistemas software en función de sus necesidades [2,5,6].

Un **estilo arquitectónico** caracteriza una familia de sistemas que están relacionados por compartir propiedades estructurales y funcionales.

Cada estilo arquitectónico proporciona un marco para construir un tipo de arquitectura que tiene ciertas propiedades. Tal arquitectura tendrá sentido aplicarla a ciertos contextos.

En las siguientes secciones se presentan algunos de los estilos arquitectónicos más utilizados en la práctica.

#### 4.2. Arquitectura en capas

La arquitectura en capas es uno de los estilos más utilizados en el desarrollo de software. En este enfoque, el sistema se organiza en distintos niveles jerárquicos, donde cada capa tiene una responsabilidad específica y se comunica únicamente con las capas adyacentes.

La **arquitectura en capas** organiza el sistema en niveles, donde cada capa proporciona servicios a la capa superior y utiliza los servicios de la capa inferior.

Esta organización permite separar responsabilidades y estructurar el sistema de manera clara. Por ejemplo, en una aplicación típica podemos distinguir capas como presentación, lógica de negocio y acceso a datos.

Una característica fundamental de este estilo es que cada capa oculta su implementación interna y expone una interfaz bien definida, lo que facilita la modularidad y el mantenimiento del sistema.

Entre sus ventajas destacan:

- Separación clara de responsabilidades
- Facilidad de mantenimiento
- Posibilidad de sustituir o modificar capas de manera independiente

Sin embargo, también presenta algunas limitaciones:

- Puede introducir sobrecarga en la comunicación entre capas
- Puede resultar rígida si se requieren accesos entre capas no adyacentes

A pesar de estas limitaciones, sigue siendo un modelo ampliamente utilizado, especialmente en aplicaciones empresariales.

### 4.3. Microkernel

La **arquitectura de microkernel** tiene como objetivo permitir añadir funcionalidad a una aplicación a través de extensiones cuya instalación no requiere modificar el núcleo del sistema. Estas extensiones habitualmente se conocen como plug-ins, y por ello la arquitectura también se conoce como **arquitectura basada en plug-ins**.

Algunos ejemplos de sistemas que utilizan esta arquitectura son:

- Entornos de programación como Eclipse o Visual Studio Code.
- Navegadores
- Sistemas operativos

La arquitectura tiene dos tipos de componentes principales:

- **Núcleo**. El núcleo del sistema implementa la funcionalidad básica (y estable, en el sentido de que no cambia) del sistema.

También ofrece una API para que las extensiones puedan instalarse e interactuar con el núcleo.

- **Plug-ins**. Un plug-in es un módulo que extiende a la aplicación principal con nueva funcionalidad.

En general, los plug-ins son independientes para que el sistema de extensiones sea simple, pero hay aplicaciones que permiten establecer dependencias entre plug-ins como por ejemplo Eclipse (a través de OSGi).

La figura [Figura 2](#) muestra los elementos de una arquitectura de Microkernel. El núcleo es extendido por **plug-ins** (relación extiende). Para realizar esta extensión el plug-in incluye dos elementos: a) un manifiesto que contiene la información de configuración acerca de cómo se realizará esta extensión y b) el código con la implementación de la extensión. Este código debe implementar las interfaces requeridas por el núcleo (para que éste pueda dirigir al plug-in) y utiliza servicios del núcleo que están disponibles a través de una API (para que el plug-in pueda pedirle al núcleo que haga ciertas tareas). Por último, el núcleo utiliza el manifiesto para saber cómo cargar el plug-in y cómo integrarlo.

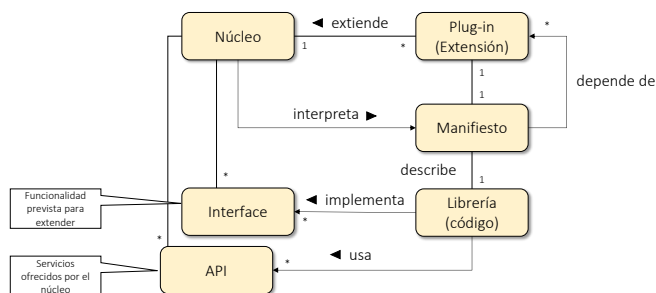


Figura 2: Elementos un arquitectura de Microkernel en un lenguaje orientado a objetos

Un requisito importante para poder implementar un sistema de este tipo es que debe ser posible cargar código dinámicamente, ya que el núcleo y los plug-ins se compilan por separado.

#### 4.4. Pipes and filters

Este estilo arquitectónico está orientado a sistemas que deben procesar flujos de datos de manera que el procesamiento se organiza como una secuencia de etapas independientes (*filters*) que se conectan a través de tuberías (*pipes*).

Cada filtro realiza una transformación sobre los datos y los pasa al siguiente filtro en la cadena. Este enfoque permite construir sistemas modulares y reutilizables, donde cada componente tiene una función bien definida.

En este estilo arquitectónico los componentes son los filtros. Cada filtro enriquece, refina o transforma sus datos de entrada produciendo un flujo de salida. La interfaz de cada filtro consta de un conjunto de puertos de entrada y un conjunto de puertos de salida. En un *pipeline* simple los filtros tienen un único puerto de entrada y un único puerto de salida, pero no tiene por que ser necesariamente siempre así. Los conectores de este estilo arquitectónico son los *pipes*. Dependiendo del tipo de sistema, estos conectores pueden funcionar en streaming (p.ej., las tuberías de UNIX) o en batch (p.ej., un compilador).

Es común en sistemas de procesamiento de datos, compiladores o herramientas de línea de comandos, donde se encadenan operaciones para obtener un resultado final.

Las principales ventajas de este estilo arquitectónico son:

- El mecanismo para organizar el sistema es la **composición funcional**. Es decir, los filtros se comportan como funciones, en el sentido matemático del término.
- Se mejora la reutilización. Por un lado, los filtros reutilizables en sistemas con el mismo formato de datos. Por otro lado, los filtros reemplazables por otros que lean el mismo formato de datos.

- Los componentes están débilmente acoplados ya que son independientes del resto de componentes.

Algunas desventajas son:

- Al componer el pipeline, el tiempo final de procesamiento puede depender del filtro más lento.
- No es habitual utilizar esta arquitectura para sistemas interactivos puesto que el tiempo de procesamiento suele ser alto.
- Puesto que los componentes no modifican los datos, existe cierta duplicación del trabajo puesto que hay que copiar los datos en la salida.

#### **4.5. Arquitectura basada en eventos**

En una arquitectura basada en eventos, los componentes del sistema no se comunican directamente entre sí, sino a través de eventos que se generan y procesan de manera asíncrona.

En lugar de invocar explícitamente a otros componentes, un módulo emite eventos que pueden ser gestionados por otros módulos interesados. Este enfoque permite diseñar sistemas desacoplados, donde los componentes pueden evolucionar de manera independiente.

Este tipo de arquitectura es especialmente útil en sistemas distribuidos, aplicaciones en tiempo real y entornos donde la escalabilidad es un requisito fundamental.

##### **4.5.1. Publicar y suscribir**

**Pendiente**

##### **4.5.2. Colas de mensajes**

**Pendiente**

#### **4.6. Arquitectura cliente-servidor**

La arquitectura cliente-servidor es un modelo en el que el sistema se divide en dos tipos de componentes: clientes, que realizan peticiones, y servidores, que proporcionan servicios. El funcionamiento general es el siguiente:

- Un servidor está esperando conexiones.
- Uno o más clientes se conectan al servidor y le envían peticiones (esto es, la conexión es muchos-a-uno).
- Las peticiones contienen información sobre el cliente, de manera que el servidor sabe a quién responder.
- La comunicación es habitualmente síncrona, lo que significa que el cliente espera a que el servidor responda antes de continuar con su trabajo. Cuando la comunicación es asíncrona, el cliente puede continuar trabajando mientras espera la respuesta del servidor (pero entonces la carga para el servidor es mayor que tiene que recordar el estado de cada cliente).

En este enfoque, los clientes no necesitan conocer la implementación interna del servidor, sino únicamente la interfaz a través de la cual interactúan con él. Esto permite centralizar la gestión de recursos y facilitar el acceso a los servicios desde múltiples clientes.



Este modelo es la base de la mayoría de las aplicaciones web actuales, donde los navegadores actúan como clientes y los servidores gestionan la lógica de negocio y el acceso a datos.

#### 4.7. Microservicios

La arquitectura de microservicios organiza el sistema como un conjunto de servicios pequeños e independientes, cada uno con una responsabilidad específica.

Estos servicios se comunican entre sí a través de interfaces bien definidas, normalmente mediante APIs. Cada servicio puede desarrollarse, desplegarse y escalarse de forma independiente.

Este enfoque es ampliamente utilizado en aplicaciones modernas en la nube, ya que facilita la escalabilidad y mejora la resiliencia del sistema. Sin embargo, también introduce complejidad en la gestión de la comunicación y el despliegue de los distintos servicios.

### 5. Arquitectura hexagonal

La arquitectura hexagonal fue propuesta por Alistair Cockburn en 2005, que la ha ido refinando hasta su versión actual denominada “Arquitectura de puertos y adaptadores” (Ports & Adapters). La última versión está documentada en [7].

El objetivo que pretende conseguir esta arquitectura es aislar el núcleo de la aplicación (la lógica de negocio) de las tecnologías externas (bases de datos, interfaces de usuario, servicios externos, etc.), para ello hay que diseñar la aplicación para que cumpla con las siguientes características:

- Pueda ejecutarse sin interfaz de usuario.
- Pueda ejecutarse sin base de datos.
- Pueda ser controlada por tests automáticos o scripts.
- Sea posible permitir cambiar tecnologías sin tocar el dominio.

La [Figura 3](#) muestra el objetivo de la arquitectura hexagonal, que es aislar el núcleo de la aplicación de las tecnologías externas. La aplicación establece un límite claro dentro del cual (el hexágono) se encuentra la lógica de negocio, mientras que las tecnologías externas se encuentran fuera del hexágono. La aplicación se comunica con el exterior a través de interfaces bien definidas (puertos) y adaptadores que traducen las llamadas entre la aplicación y el exterior. De esta manera, se puede desarrollar y probar la lógica de negocio de manera independiente de las tecnologías externas, lo que facilita el desarrollo incremental e independiente, la creación de pruebas y la evolución del sistema a lo largo del tiempo.

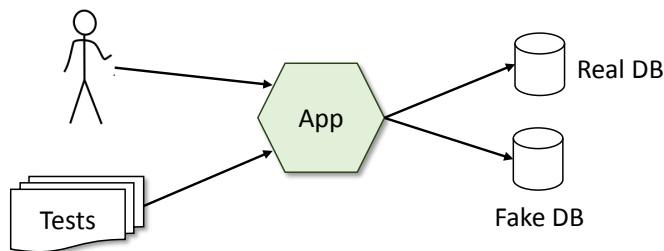


Figura 3: Objetivo de la arquitectura hexagonal: aislar el núcleo de la aplicación de las tecnologías externas.

## 5.1. Elementos

La arquitectura tiene cuatro elementos básicos:

- La aplicación o sistema en sí (lo que está dentro del hexágono).
- Los puertos que definen la interfaz de comunicación entre la aplicación y el exterior.
- Los actores externos que dirigen la aplicación o son dirigidos por la aplicación.
- Los adaptadores que se encargan de traducir las llamadas entre la aplicación y los actores externos a través de los puertos.

Además, desde un punto de vista práctico debe haber un elemento adicional que configure la conexión entre los cuatro elementos anteriores, pero eso está fuera de la arquitectura propiamente dicha puesto que es un elemento de infraestructura.

En esta arquitectura un **aplicación** se ve como un componente reutilizable que no tiene dependencias tecnológicas externas. Esto quiere decir que esencialmente tiene lógica de negocio y lógica de coordinación (servicios de aplicación), pero no tiene lógica de infraestructura (bases de datos, interfaces de usuario, servicios externos, etc.)<sup>1</sup>.

Para comunicarse con el exterior la aplicación define **puertos** (ports) que son interfaces que definen cómo la aplicación interactúa con el exterior. Los puertos son de dos tipos: puertos de entrada (**driving ports**) y puertos de salida (**driven ports**). Los puertos de entrada son los que utilizan los actores externos para interactuar con la aplicación, mientras que los puertos de salida son los que utiliza la aplicación para interactuar con los actores externos.

Esencialmente un puerto declara una o más interfaces que definen cómo la aplicación interactúa con el exterior. Hay dos tipos de interfaces: provided interfaces y required interfaces.

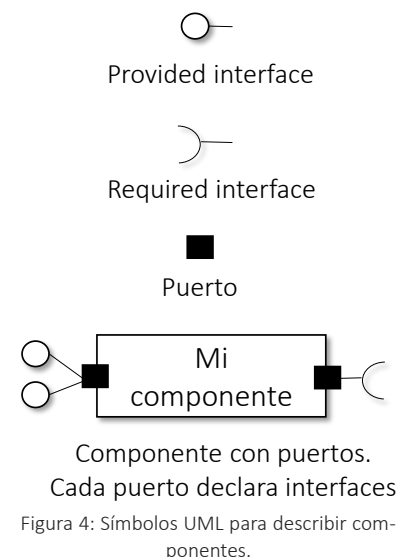
- La provided interface es la interfaz que la aplicación ofrece al exterior para que el pueda “dirigir” la aplicación.
- La required interface es la interfaz que la aplicación requiere del exterior para que la aplicación pueda “conducir” ciertos elementos.

### ⚠ ¿Quién define las interfaces?

La aplicación define ambos tipos de interfaces puesto las interfaces representan el contrato que establece con el exterior. La diferencia es que las “provided interfaces” son el contrato que la aplicación se compromete

Esencialmente cualquier elemento físico con el que interacciona la aplicación está fuera de la aplicación. Esto incluye bases de datos, interfaces de usuario, servicios externos, redes, etc.

<sup>1</sup>La aplicación puede ser fácilmente distribuida como una librería.



a cumplir, mientras que las “required interfaces” son el contrato que la aplicación necesita o exige que el exterior cumpla.

Para interactuar con la aplicación se crean **adaptadores**. Un adaptador es un componente que implementa una interfaz de puerto (provided o required) y se encarga de transformar las llamadas entre la aplicación y el actor externo.

Algunos ejemplos de adaptadores para los puertos conducidos son:

- Un adaptador de base de datos que implementa la required interface del puerto de salida para permitir a la aplicación almacenar y recuperar datos de una base de datos ej., MySQL.
- Un adaptador de servicio web que implementa la required interface del puerto de salida para permitir a la aplicación comunicarse con una API externa (ej., una API de un LLM).

Estos adaptadores se implementan fuera de la aplicación en sí pero se despliegan junto con la aplicación.

Algunos ejemplos de adaptadores para los puertos dirigidos son:

- Una interfaz de usuario (UI) que invoca la provided interface del puerto de entrada para permitir a un usuario interactuar con la aplicación.
- Un conjunto de pruebas automatizadas que invoca la provided interface del puerto de entrada para probarlas

Es importante observar que la implementación concreta de la “provided interface” **está dentro** de la aplicación. El adaptador no la conoce porque se comunica a través de la interfaz. Su labor es traducir las llamadas del actor externo a la interfaz de la aplicación.

## 5.2. Ejemplo de arquitectura hexagonal y DDD

Supongamos que se quiere implementar una aplicación de pedidos para una tienda online. La funcionalidad principal es la de crear un pedido y cuando esté listo procesar el pago y guardar el pedido en una base de datos. Para ello necesita:

- Implementar el modelo de dominio que representa los pedidos y la lógica de negocio asociada.
- Definir un puerto de entrada para la operación de crear pedido, que será invocado por una interfaz de usuario o por pruebas automatizadas.
- Definir puertos de salida para interactuar con la base de datos para guardar los pedidos y para interactuar con un servicio de pago externo para procesar los pagos.
- Implementar adaptadores para la base de datos (ej., Postgresql) y para el servicio de pago (ej., Stripe).
- Implementar una interfaz de usuario (ej., una API REST) que invoque el puerto de entrada para crear pedidos.

### 5.2.1. Diseño e implementación de la aplicación con DDD

En la arquitectura hexagonal para el modelo de dominio se puede seguir el enfoque que se desee. Sin embargo, en nuestro caso utilizaremos Domain-

Driven Design (DDD) de manera que el dominio se organizará según los patrones tácticos de DDD.

Definiremos entidades como `Pedido`, `LineaPedido`, `Cliente`, etc., junto con la lógica de negocio asociada. Además habrá que definir servicios de aplicación para representar las operaciones que pueden realizar los usuarios de la aplicación.

```
public class Pedido {  
  
    private final PedidoId id;  
    private final ClienteId clienteId;  
    private final List<LineaPedido> items;  
    private final LocalDateTime fechaCreacion;  
    private EstadoPedido estado;  
  
    // Resto del código  
    public void addLineaPedido(ProductoId productoId, int qty) {  
        // Lógica para añadir una línea de pedido  
    }  
  
    public void marcarComoPagado() {  
        // Lógica para marcar el pedido como pagado  
    }  
  
}
```

Para poder realizar pagos necesitamos interactuar con un **servicio de pago externo**. Este es un servicio de infraestructura, para el cual definiremos una interfaz dentro de la aplicación pero que será implementada por un adaptador externo. Este servicio se usará en el servicio de aplicación correspondiente (ver debajo).

```
/** Definición de un servicio de infraestructura */  
public interface ServicioDePago {  
  
    EstadoPago procesarPago(PedidoId pedidoId, Dinero total);  
  
}
```

El **servicio de aplicación** se encarga de coordinar la lógica de negocio para gestionar los pedidos. Tiene dos dependencias (ver constructor): el repositorio de pedidos para guardar y recuperar los pedidos de la base de datos, y el servicio de pago para procesar los pagos.

```
public class ServicioDePedidos {  
  
    // Dependencias del servicio. Durante la configuración  
    // de la aplicación se inyectarán las implementaciones  
    // concretas de estas dependencias.  
    private final RepositorioDePedidos repositorio;  
    private final ServicioDePago servicioDePago;  
  
    public ServicioDePedidos(RepositorioDePedidos repositorio,  
                             ServicioDePago servicioDePago) {  
        this.repositorio = repositorio;  
        this.servicioDePago = servicioDePago;  
    }  
  
}
```

```
// ...

@Override
public Pedido pagarPedido(String pedidoId)
    throws PagoRechazadoException {

    Pedido pedido = repositorio.findPorId(pedidoId);
    Preconditions.checkNotNull(pedido, "Pedido no encontrado");

    EstadoPago estadoPago = servicioDePago.procesarPago(pedidoId,
pedido.getTotal());

    if (estadoPago == EstadoPago.APROBADO) {
        pedido.marcarComoPagado();
        repositorio.guardar(pedido);
    } else {
        throw new PagoRechazadoException("El pago fue rechazado
para el pedido: " + pedidoId);
    }

    return pedido;
}
}
```

### 5.2.2. Puertos y adaptadores

La aplicación definida en las clases anteriores es independiente de las tecnologías externas, pero para que pueda funcionar es necesario definir los puertos y adaptadores correspondientes. Como se muestra en la Fig. Figura 5, la aplicación tiene:

Un puerto de entrada (*driving port*):

- **ServicioDePedidos** que define la interfaz para gestionar los pedidos.

Dos puertos de salida (*driven ports*):

- **RepositorioDePedidos** que define la interfaz para almacenar de manera persistente los pedidos.
- **ServicioDePago** que define la interfaz para interactuar con el servicio de pago externo

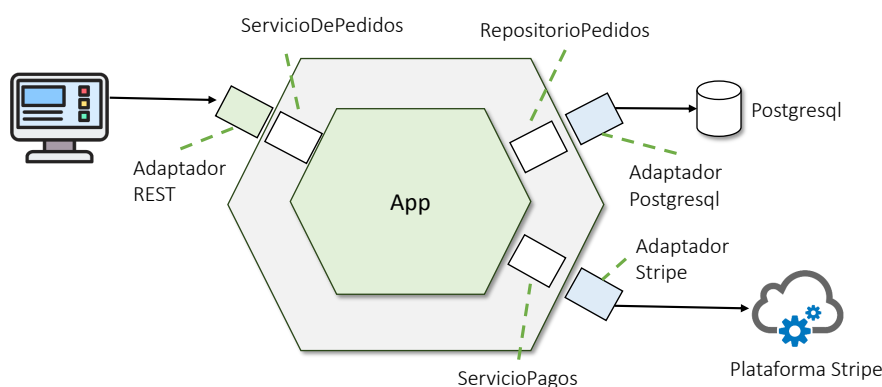


Figura 5: Ejemplo de arquitectura hexagonal cuyo nucleo es la aplicación creada con DDD para pedidos.

### 5.2.3. Correspondencia entre DDD y la arquitectura hexagonal

Los elementos de DDD tienen una correspondencia directa con los elementos de la arquitectura hexagonal:

- El modelo de dominio (entidades, objetos valor, servicios de dominio, etc.) se encuentra dentro del núcleo de la aplicación.
- Los servicios de aplicación (como `ServicioDePedidos`) también se encuentran dentro de la aplicación (“dentro del hexágono”), pero se interpretan como un puerto de entrada para que los actores externos puedan interactuar con la aplicación.
- Los repositorios corresponden a puertos de salida o *driven ports* que la aplicación utiliza para interactuar con la base de datos.
- Los servicios de infraestructura (como `ServicioDePago`) que necesite la aplicación también corresponden a puertos de salida o *driven ports*.

### 5.3. Ventajas y desventajas

Los beneficios de esta arquitectura son los siguientes:

1. **Pruebas.** Se pueden escribir y ejecutar pruebas a nivel de sistema sin necesidad de configurar o conectar a las tecnologías de producción, lo cual facilita la creación de pruebas.
2. **No hay contaminación de detalles tecnológicos.** La propia arquitectura asegura que las dependencias externas no afecten a la lógica de negocio. Esto es así porque las pruebas van a demostrar que los detalles de la UI o de tecnología (ej., base de datos) no contaminan la lógica de negocio. ¿Por qué? Porque si así fuera no podrían ejecutarse las pruebas.
3. **Desarrollo incremental e independiente.** Diferentes equipos pueden desarrollar sus partes del sistema de manera independiente, probarlas por separado y conectarlas mediante interfaces bien definidas y probadas. Además, el sistema se puede empezar a construir sin necesidad de tener todas las conexiones externas listas.
4. **Mantenibilidad:** Se pueden reemplazar las conexiones externas a medida que cambian las necesidades tecnológicas y de negocio a lo largo de los años.
5. **Diseño centrado en el dominio:** Al mantener los elementos tecnológicos fuera del núcleo de la aplicación, se puede centrar en el diseño del dominio, por ejemplo utilizando Domain-Driven Design (DDD).
6. **No hay necesidad de recompilar.** El núcleo de la aplicación no depende de las tecnologías externas, por lo que no es necesario recompilar el sistema para cambiar entre entornos de prueba y producción o para actualizar las tecnologías externas. En otras palabras el sistema es “externamente extensible” como un sistema de plugins.

Sin embargo, existen algunos costes asociados a esta arquitectura:

1. Es necesario diseñar y definir las interfaces entre el núcleo de la aplicación y las tecnologías externas, lo cual requiere tiempo y esfuerzo adicional.
2. Hay una sobrecarga inicial en la configuración del proyecto y la estructura del código para implementar la arquitectura hexagonal.
3. Puede haber una ligera disminución del rendimiento debido a la capa adicional de abstracción entre el núcleo de la aplicación y las tecnologías externas.

## 6. Arquitectura REST

REST (REpresentational State Transfer) es un estilo arquitectónico para sistemas distribuidos, especialmente para servicios web. Fue definido formalmente por Roy Fielding en su tesis doctoral en el año 2000 [8].

REST es independiente de la tecnología, pero en la práctica se suele implementar utilizando el protocolo HTTP y formatos de datos como JSON o XML para la representación de los recursos.

En esencia, un servicio REST se organiza como un sitio web en el sentido de que los recursos se identifican mediante URIs. Se accede a los recursos a través de peticiones que un cliente realiza a un servidor (para realizar las peticiones se utilizan los métodos HTTP estándar, GET, POST, PUT, DELETE, etc.). Además, los recursos pueden estar relacionados entre sí mediante enlaces (hypermedia).

Una URI (Uniform Resource Identifier) es una cadena de caracteres que identifica de manera única un recurso en la web. Por ejemplo, `https://tienda.com/productos/123` es una URI que identifica el recurso del producto con ID 123 en la tienda online. Una URL (Uniform Resource Locator) es un tipo específico de URI que no solo identifica un recurso, sino que también proporciona la información necesaria para localizarlo en la red (normalmente identifica el protocolo, el host y la ruta).

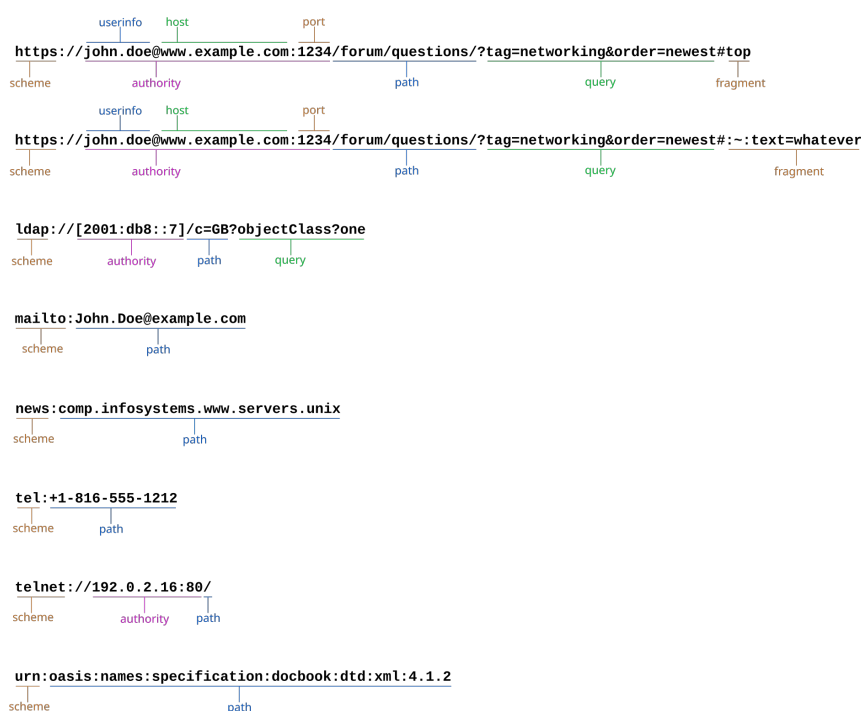


Figura 6: Ejemplos de URIs. [https://en.wikipedia.org/wiki/Uniform\\_Resource\\_Identifier](https://en.wikipedia.org/wiki/Uniform_Resource_Identifier)

Por ejemplo, en una tienda online, los productos pueden ser recursos accesibles mediante URLs como `https://tienda.com/productos/123`, y las categorías de productos pueden estar relacionadas con los productos mediante enlaces. Por ejemplo,

Ante una petición del cliente como:

```
GET https://tienda.com/productos/123
```

El servidor devolvería la representación del recurso en cierto formato, por ejemplo JSON, como se muestra a continuación:

```
{
  "id": 123,
  "nombre": "Camiseta",
  "precio": 19.99,
  "categoria": {
    "id": 5,
    "nombre": "Ropa",
    "enlace": "https://tienda.com/categorias/5"
  }
}
```

Entonces, el cliente puede realizar las acciones necesarias, incluyendo seguir el enlace a la categoría para obtener más información sobre ella.

Antes de presentar en más detalle la arquitectura REST, es importante entender el protocolo HTTP, que se presenta a continuación.

## 6.1. El protocolo HTTP

El protocolo HTTP (Hypertext Transfer Protocol) fue creado en 1989 en el CERN por [Tim Berners-Lee](#), junto con el lenguaje HTML, con el objetivo de compartir documentos científicos entre investigadores. Es la base de la World Wide Web (WWW), el sistema que hizo posible la navegación mediante enlaces (hipertexto).

HTTP permite que los clientes (como los navegadores web) se comuniquen con los servidores (ej., que alojan los sitios web) para solicitar y recibir recursos, como páginas web, imágenes, videos, etc. Para ello, HTTP define el tipo de peticiones soportadas (GET, POST, PUT, DELETE, etc.), el formato de las peticiones y respuestas, y los códigos de estado que indican el resultado de las peticiones.

La [Figura 7](#) muestra un ejemplo de la arquitectura cliente-servidor de la Web, donde un navegador (actuando como cliente) realiza una petición HTTP a un servidor para obtener un recurso (una páginas web).

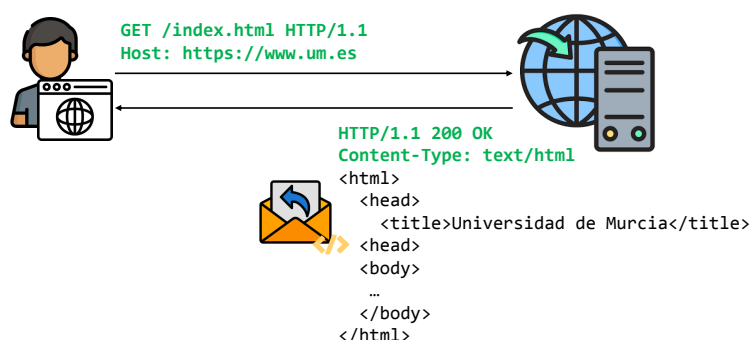


Figura 7: Ejemplo de la arquitectura cliente-servidor de la Web.



Se recomienda consultar la referencia de MDN para HTTP (<https://developer.mozilla.org/es/docs/Web/HTTP>). En particular, los métodos HTTP (<https://developer.mozilla.org/es/docs/Web/HTTP/Methods>).

### 6.1.1. Peticiones y respuestas

La comunicación entre cliente y servidor se realiza mediante peticiones HTTP. Las peticiones HTTP incluyen un método (GET, POST, PUT, DELETE, etc.), una URL que identifica el recurso, encabezados que proporcionan metadatos adicionales y, opcionalmente, un cuerpo que contiene datos.

Una **petición** (Request) es una solicitud enviada por el cliente al servidor para realizar una acción específica sobre un recurso.

Una **respuesta** (Response) es la información que el servidor envía de vuelta al cliente en respuesta a una petición. La respuesta incluye un código de estado HTTP, encabezados y un cuerpo con los datos solicitados o el resultado de la acción.

### 6.1.2. Métodos HTTP

HTTP está basado en operaciones estandarizadas que indican la acción sobre un recurso. Estas operaciones se denominan **métodos HTTP**. Así, cada petición indica el método que se desea ejecutar sobre el recurso identificado por la URL. Cada método tiene una semántica específica que define su comportamiento y efectos sobre los recursos.

Los métodos están basados en las operaciones CRUD (Create, Read, Update, Delete), que son las operaciones básicas para gestionar recursos en un sistema, aunque también hay algunos métodos de diagnóstico.

A continuación se muestra una tabla con los métodos HTTP más comunes.

| Método | Descripción                 | Efecto lateral | Idempotente |
|--------|-----------------------------|----------------|-------------|
| GET    | Obtener información         | No             | Sí          |
| POST   | Crear un recurso            | Sí             | No          |
| PUT    | Reemplazar recurso completo | Sí             | Sí          |
| PATCH  | Modificar parcialmente      | Sí             | No          |
| DELETE | Eliminar recurso            | Sí             | Sí          |

#### Método GET

Se utiliza para la lectura de recursos. Un método GET nunca cambia el estado del recurso, es decir, no tiene efectos laterales. Puesto que es idempotente, la respuesta a una petición GET puede ser cacheada por el cliente o por intermediarios (proxies) para mejorar el rendimiento y reducir la carga en el servidor.

Ejemplos de peticiones GET (formato informal):

```
GET /empleados           # Listar todos los empleados
GET /empleados/1         # Obtener los detalles del empleado 1
```

#### Semántica de las operaciones

Se dice que una operación tiene **efectos laterales** (*side effects*) cuando la ejecución de la operación modifica el estado del servidor, esto es, si añade, modifica o elimina un recurso.

Un método HTTP es **idempotente** cuando se puede ejecutar la misma petición repetidamente y el estado resultante es el mismo.

Algunos ejemplos de idempotencia son:

- $f(x) = x$   
porque  $f(f(x)) = f(x) = x$
- Multiplicar por 1 por que  $1 * 1 * 1 = 1 * 1 = 1$
- Botón de un ascensor: pulsar varias veces el botón no cambia el estado del ascensor (no hará que llegue antes).

## Método POST

Se utiliza para crear recursos. Cada invocación de POST crea un nuevo recurso y por tanto no es idempotente.

```
POST /empleados # Crea un nuevo empleado.

# La respuesta incluirá una cabecera Location con la URL del nuevo
recurso creado, por ejemplo:
HTTP/1.1 201 Created
Location: /empleados/12
```

## Método PUT

Se utiliza para crear o actualizar (reemplazar) un recurso. Por tanto, en la petición hay que incluir la descripción completa del recurso en la petición. A diferencia de PATCH, PUT no está pensado para actualizaciones parciales. Al contrario que POST, el método PUT sí que es idempotente.

```
PUT /empleados/1 # actualiza el empleado 1
PUT /empleados/1 # puede significar crear el empleado 1 si no
existe (menos común)
```

La diferencia entre el método PUT y el método POST es que PUT sí es un método idempotente: llamarlo una o más veces de forma sucesiva tiene el mismo efecto (sin efectos secundarios), mientras que una sucesión de peticiones POST idénticas pueden tener efectos adicionales, como enviar una orden varias veces.

Entonces, ¿Cómo puede ser que PUT sea idempotente cuando se usa para crear recursos? ¿Qué impacto tiene esto en la práctica?

La razón es que PUT debe incluir el identificador del recurso en la URL. Eso significa que si PUT se utiliza para crear recursos se está asumiendo que el cliente conoce la manera de generar el identificador del recurso, lo cual no es común, ya que normalmente el servidor es el encargado de generar los identificadores.

### 6.1.3. Formato de los mensajes

En HTTP, tanto las peticiones como las respuestas siguen un formato específico. En <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages> se puede encontrar una descripción detallada del formato de los mensajes HTTP.

Tanto las peticiones como las respuestas comparten una estructura similar (ver Fig. Figura 8) que consta de los siguientes componentes:

- **start-line:** Es la primera línea del mensaje y contiene información sobre el método HTTP (en el caso de las peticiones) o el código de estado (en el caso de las respuestas).
- **headers:** Son líneas adicionales que proporcionan metadatos sobre el mensaje, como el tipo de contenido, la longitud del cuerpo, las cookies, etc.
- **empty line:** Es una línea vacía que indica el final de los encabezados y el comienzo del cuerpo del mensaje.
- **body:** Es la parte del mensaje que contiene los datos asociados a la petición o respuesta. Por ejemplo, en una petición POST, el cuerpo contiene los datos que se quieren enviar al servidor, mientras que en una respuesta, el cuerpo contiene la representación del recurso solicitado. En el caso de una petición GET, el cuerpo está vacío, ya que no se envían datos al servidor.

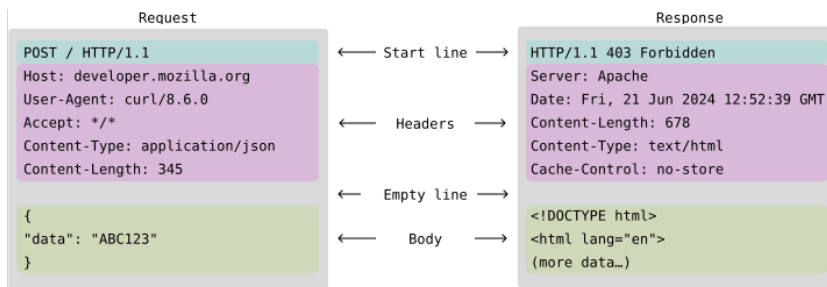


Figura 8: Formato de los mensajes HTTP. Extraído de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>

HTTP ofrece **negociación de contenido** para que el cliente y el servidor puedan acordar el formato de los datos que se intercambian. Para ello:

- El cliente indica el formato de los datos que está enviando al servidor mediante el encabezado **Content-Type** en las peticiones que incluyen un cuerpo (como POST o PUT). Por ejemplo, si el cliente está enviando datos en formato JSON, incluirá el encabezado **Content-Type: application/json**.
- Además, el cliente puede incluir en los encabezados de la petición un campo **Accept** que indique los formatos de datos que puede procesar (por ejemplo, **application/json**, **text/html**, etc.).
- El servidor puede responder con un encabezado **Content-Type** que indique el formato de los datos que está enviando en el cuerpo de la respuesta.

#### 6.1.4. Códigos de estado HTTP

HTTP estandariza los códigos de respuesta que el servidor envía al cliente para indicar el resultado de la petición. Estos códigos de estado son números de tres dígitos que se agrupan en diferentes categorías según su significado.

- **2xx (éxito)**: Significan que la petición ha sido procesada con éxito.
  - **200 OK**: La petición se ha procesado correctamente y se devuelve la información solicitada.
  - **201 Created**: La petición se ha procesado correctamente y se ha creado un nuevo recurso.
  - **204 No Content**: La petición se ha procesado correctamente pero no hay contenido que devolver.
- **3xx (redirección)**: Indican que el recurso solicitado está en otro lugar y el cliente debe realizar una nueva petición a la URL proporcionada en la respuesta u obtener el recurso por otro medio (ej., cuando está en caché).
  - **301 Moved Permanently**: El recurso solicitado ha sido movido permanentemente a una nueva URL.
  - **304 Not Modified**: El recurso solicitado no ha sido modificado desde la última vez que se solicitó, por lo que el cliente puede usar la versión en caché.
- **4xx (error del cliente)**: Indican que hubo un error en la petición realizada por el cliente. Esto puede deberse a una solicitud mal formada, falta de autorización, recurso no encontrado, etc.
  - **400 Bad Request**: La petición no se pudo entender o procesar debido a una petición mal formada.

- ▶ **401 Unauthorized**: La petición requiere autenticación y el cliente no ha proporcionado credenciales válidas.
- ▶ **403 Forbidden**: El servidor entiende la petición pero se niega a autorizarla.
- ▶ **404 Not Found**: El recurso solicitado no se encontró en el servidor.
- **5xx (error del servidor)**: Indican que hubo un error en el servidor al procesar la petición. Esto puede deberse a un error interno, una sobrecarga, un tiempo de espera agotado, etc.
  - ▶ **500 Internal Server Error**: El servidor encontró una condición inesperada que le impidió cumplir con la petición.
  - ▶ **503 Service Unavailable**: El servidor no está disponible temporalmente, generalmente debido a mantenimiento o sobrecarga.

Entender los códigos de estado HTTP más comunes es fundamental para desarrollar aplicaciones web y servicios RESTful, ya que permiten a los desarrolladores manejar las respuestas del servidor de manera adecuada y proporcionar una mejor experiencia al usuario.

## 6.2. La arquitectura REST

La arquitectura REST se basa en los mismos principios de la WWW, pero en los servidores en lugar de servir páginas web (ficheros HTML), sirven recursos representados en algún formato estructurado como JSON o XML.

De acuerdo a [8], REST se basa en seis restricciones arquitectónicas fundamentales:

1. Arquitectura cliente-servidor
2. Sin estado (Stateless)
3. Cache
4. Interfaz uniforme
5. Organización en capas
6. Código bajo demanda (opcional)

Estas restricciones definen las reglas y principios que deben seguirse para diseñar sistemas RESTful. A continuación se describen cada una de estas restricciones.

### 6.2.1. Arquitectura cliente-servidor

El sistema debe tener dos tipos de componentes principales: clientes y servidores, es decir, se debe seguir el patrón arquitectónico cliente-servidor.

La principal razón es la **separación de responsabilidades** de manera que el cliente es responsable de la interfaz de usuario y la experiencia del usuario, mientras que el servidor maneja la lógica de negocio y el almacenamiento de datos. Esta separación permite una mayor escalabilidad y flexibilidad en el desarrollo y despliegue de aplicaciones.

En la Fig. [Figura 9](#) se muestra como tres clientes diferentes pueden acceder a un mismo servicio REST. Para que esto pueda funcionar es importante que la interfaz entre los clientes y el servidor esté bien definida y se mantenga estable (es decir, si se cambia la interfaz los clientes pueden verse afectados)

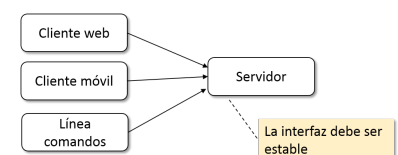


Figura 9: Clientes y servidores en la arquitectura REST

### 6.2.2. Sin estado (Stateless)

Cada petición del cliente al servidor debe contener toda la información necesaria para que el servidor pueda entender y procesar la petición. Es decir, el estado de la sesión debe almacenarse totalmente en el cliente.

El servidor no debe almacenar ningún estado del cliente entre peticiones. Por “estado del cliente” se refiere a información sobre las peticiones e interacciones previas del cliente. Por ejemplo, en un proceso de compra no se almacenará la etapa del proceso o información de autenticación del cliente en el servidor, sino que toda esa información se incluirá en cada petición que el cliente realice al servidor. Esto no quiere decir que el servidor no almacene estado: el estado que almacena el servidor es el estado de los recursos, pero no el estado de las interacciones con los clientes.

Hay tres razones importantes para esta restricción:

1. **Visibilidad.** A la hora de monitorizar y depurar el sistema, es más sencillo porque no es necesario considerar el estado del servidor, es suficiente con analizar las peticiones y respuestas.
2. **Fiabilidad.** Si el servidor no mantiene estado, es más sencillo recuperarse de fallos, ya que cualquier instancia del servidor puede manejar cualquier petición sin necesidad de tener en cuenta el estado previo.
3. **Escalabilidad.** Al no tener que mantener el estado de los clientes, el servidor puede manejar un mayor número de peticiones simultáneas y distribuir la carga entre múltiples servidores de manera más eficiente.

Una pregunta muy relevante es cómo es posible conseguir que el servidor no tenga estado cuando muchas operaciones requieren mantener un estado, como por ejemplo el *login* o el carrito de la compra en una tienda online<sup>2</sup>.

### 6.2.3. Cache

El servidor puede marcar las respuestas como cacheables o no cacheables. Si una respuesta es cacheable, el cliente puede almacenar la respuesta y reutilizarla para peticiones futuras, evitando así tener que contactar con el servidor nuevamente para obtener el mismo recurso.

La principal cuestión es cómo determinar en qué momento expira la cache para evitar que el cliente utilice datos obsoletos.

Por ejemplo, en Spring Boot podríamos utilizar dos mecanismos para controlar la cache de las respuestas: `Cache-Control` y `ETag`. El siguiente ejemplo muestra cómo utilizar ambos mecanismos en una respuesta a una petición GET:

```
@GetMapping("/book/{id}")
public ResponseEntity<Book> showBook(@PathVariable Long id) {

    Book book = findBook(id);
    String version = book.getVersion();

    return ResponseEntity
        .ok()
```

**Tesis de Roy Fielding, Sección 5.1.3:** Like most architectural choices, the stateless constraint reflects a design trade-off. The disadvantage is that it may decrease network performance by increasing the repetitive data (per-interaction overhead) sent in a series of requests, since that data cannot be left on the server in a shared context. In addition, placing the application state on the client-side reduces the server's control over consistent application behavior, since the application becomes dependent on the correct implementation of semantics across multiple client versions.

<sup>2</sup><https://chatgpt.com/share/6943dbab-dca8-8010-8e3a-07397edbbd40>

```
.cacheControl(CacheControl.maxAge(30, TimeUnit.DAYS))  
.eTag(version)  
.body(book);  
}
```

En este ejemplo:

- **Cache-Control: max-age=2592000** indica que la respuesta es cacheable durante 30 días. El navegador no pediría de nuevo

el libro durante ese tiempo, a menos que el usuario borre la cache o se utilice un mecanismo de actualización forzada (como refrescar la página CTRL+SHIFT+R). Este mecanismo es útil para recursos que no cambian con frecuencia, como imágenes o archivos estáticos.

- **ETag: "version"** proporciona una etiqueta única para la versión del recurso. El cliente puede incluir esta etiqueta en una petición futura utilizando el encabezado **If-None-Match**. Para esto, el servidor debe tener una manera de generar

versiones de los recursos, por ejemplo utilizando un campo de versión en la base de datos o un hash del contenido del recurso. El funcionamiento (asumiendo solo el ETag) es como sigue:

- El Cliente pide el libro.
- Recibe el JSON, con un ETag: "v1".
- El Cliente guarda el ETag en caché.
- En la base de datos el libro cambia, y pasa a tener la versión "v2".
- El Cliente vuelve a pedir el libro:
- Su navegador envía el header If-None-Match: "v1".
- El servidor compara "v1" con la versión actual del libro "v2". La condición falla y el servidor envía el nuevo JSON con el nuevo ETag.

#### 6.2.4. Interfaz uniforme

El principio de **interfaz uniforme** establece que la interfaz entre los clientes y el servidor debe seguir un conjunto de reglas y convenciones que permitan a los clientes interactuar con el servidor de manera consistente y predecible.

Por tanto, cada recurso de una API REST debe seguir una estructura predecible y estandarizada. Para ello, lo habitual en un API REST es hacer uso de HTTP para conseguir esta uniformidad, siguiendo las siguientes reglas:

1. Identificación de recursos: Cada recurso debe ser identificado mediante una URI.
2. Uso de métodos HTTP estandar: La interacción con los recursos se hace utilizando la semántica proporcionada

por los métodos HTTP (GET, POST, PUT, DELETE, etc.)<sup>3</sup>.

3. Mensajes auto-descriptivos: Cada mensaje (petición o respuesta) debe contener toda la información necesaria para que el receptor pueda entenderlo y procesarlo correctamente, sin necesidad de información adicional.
4. HATEOAS (Hypermedia as the Engine of Application State): El servidor debe incluir hipervínculos en las respuestas que permitan a los

<sup>3</sup>Observe que esta aproximación es diferente a otras aproximaciones donde el desarrollador define un conjunto de métodos específicos lo que hace la interfaz menos predecible.

clientes descubrir y navegar por los recursos disponibles de manera dinámica, sin necesidad de conocer la estructura completa de la API de antemano.

La principal desventaja de esta aproximación es que puede resultar menos eficiente, ya que la uniformidad de la interfaz hace que la interacción no siempre utilice el mecanismo más óptimo para caso específico.

#### 6.2.5. Organización en capas

El sistema debe estar organizado en capas jerárquicas, de manera que un cliente no puede conocer qué hay detrás del servidor con el que se está comunicando.

Con esta organización el cliente no puede saber si está comunicandose directamente con el servidor final o con un intermediario (proxy, gateway, etc.). Cada capa tiene una función específica y puede interactuar solo con las capas adyacentes.

Esta decisión arquitectural tiene varias ventajas:

1. Extensibilidad: Se pueden añadir nuevas capas o modificar las existentes sin afectar a otras partes del sistema **siempre que se mantenga la misma interfaz**.
2. Seguridad: Las capas intermedias pueden actuar como filtros o firewalls, protegi
3. Escalabilidad: Un servidor puede distribuir la carga entre múltiples servidores intermedios para mejorar el rendimiento y la disponibilidad.

#### 6.2.6. Código bajo demanda (opcional)

El servidor puede enviar código ejecutable (como scripts JavaScript) al cliente para que este lo ejecute. Esto permite extender la funcionalidad del cliente de manera dinámica sin necesidad de actualizar el software del cliente.

### Bibliografía

- [1] A. H. Eden y R. Kazman, «Architecture, design, implementation», en *25th International Conference on Software Engineering, 2003. Proceedings.*, 2003, pp. 149-159.
- [2] M. Richards, «Software Architecture Patterns», 2022.
- [3] R. Sethi, *Software Engineering: Basic Principles and Best Practices*. Cambridge University Press, 2022.
- [4] M. T. Valente, «Software Engineering: A Modern Approach». [En línea]. Disponible en: <https://softengbook.org/>
- [5] M. Cristiá, «Introducción a la Arquitectura de Software», *Research-Gate.[Online]*. Recuperado de: [https://www.researchgate.net/publication/251932352\\_Introduccion\\_a\\_la\\_Arquitectura\\_de\\_Software](https://www.researchgate.net/publication/251932352_Introduccion_a_la_Arquitectura_de_Software), 2008.
- [6] M. Cristiá, «Catálogo Incompleto de Estilos Arquitectónicos», *Cátedra Ing. Software Universidad Nacional de Rosario*, 2006.
- [7] A. Cockburn y J. M. G. de Paz, «Hexagonal architecture explained», *How the ports & adapters architecture simplifies your life, and how to implement it. Humans and Technology Inc*, 2024.
- [8] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.